

Husky Pre-commit Hooks Implementation

Husky Pre-commit Hooks Implementation

Overview

This document explains our **local git hooks** implementation using Husky, transformed from Chris Hayuk’s expert Python pre-commit patterns to TypeScript/Next.js.

Purpose: Catch issues FAST (2-60 seconds) before they enter version control, providing immediate feedback and auto-fixes.

Expert Patterns Implemented

Based on Chris Hayuk’s [vibe-coding-templates](#):

Pattern	Python (Original)	TypeScript (Our Implementation)	Stage
Linting	ruff check --fix	eslint --fix	pre-commit
Formatting	ruff format	prettier --write	pre-commit
Type Checking	mypy --strict	tsc --noEmit	pre-commit
Full Lint	ruff check .	pnpm run lint	pre-push
Build Check	uv build	pnpm run build	pre-push
Security Audit	bandit/safety	pnpm audit	pre-push
Secret Detection	detect-secrets	Basic grep patterns	pre-push
Tests (TODO)	pytest -m "not slow"	pnpm run test:run	pre-push
Coverage (TODO)	pytest --cov --cov-fail-under=80	vitest --coverage	pre-push

How It Works

Pre-commit Hook (Fast: 2-10 seconds)

Runs on `git commit`:

1. Lint-staged → ESLint `--fix` + Prettier (ONLY staged files)
2. Type check → TypeScript strict mode (entire project)

Why these checks? - **Fast:** Only lints/formats staged files, not entire codebase - **Auto-fix:** Automatically fixes linting and formatting issues - **Type safety:** Catches type errors before they reach CI/CD - **Fail-fast:** Blocks commit if type errors exist

Pre-push Hook (Comprehensive: 30-60 seconds)

Runs on `git push`:

1. Full lint → All files (informational warnings)
2. Build check → Vite build (catches build errors)
3. Security audit → pnpm audit (catches vulnerabilities)
4. Secret detection → Basic pattern matching (.env, .key, .pem files)
5. [TODO] Test suite → Run all tests
6. [TODO] Coverage → Ensure 80% minimum coverage

Why these checks? - **Comprehensive:** Checks entire codebase, not just staged files - **Build safety:** Ensures code builds before pushing - **Security-first:** Catches known vulnerabilities early - **Secret prevention:** Warns about potential secret leaks - **Quality gates:** Future test/coverage checks

File Structure

```
.husky/  
├── _/                # Husky internal files  
├── pre-commit        # Fast local checks (2-10s)  
└── pre-push          # Comprehensive checks (30-60s)  
  
package.json          # lint-staged configuration
```

Configuration

`package.json` - lint-staged

```
"lint-staged": {  
  "*.ts,tsx,js,jsx": [  
    "eslint --fix",  
    "prettier --write"  
  ],  
}
```

```

    ".*.{json,md,yaml,yml}": [
      "prettier --write"
    ]
  }
}

```

What this does: - **TypeScript/JavaScript:** Auto-fix linting issues, format code - **JSON/Markdown/YAML:** Auto-format

Usage

Normal Workflow (hooks run automatically)

```

# Stage your changes
git add .

# Commit (pre-commit hook runs automatically)
git commit -m "feat: Add new feature"
# → Linting + formatting staged files...
# → Type checking...

# Push (pre-push hook runs automatically)
git push
# → Full lint...
# → Build check...
# → Security audit...
# → Secret detection...

```

Skip Hooks (emergency use only)

```

# Skip pre-commit hook
git commit -m "WIP: Quick save" --no-verify

# Skip pre-push hook
git push --no-verify

```

Warning: Only use --no-verify in emergencies! Skipped checks will still fail in CI/CD pipeline.

Installation (for new developers)

```

# 1. Clone repository
git clone <repo-url>

# 2. Install dependencies (automatically runs husky setup via "prepare" script)
pnpm install

# 3. Hooks are ready! No manual setup needed.

```

Learning from Chris Hayuk's Patterns

Pattern 1: Multi-Stage Strategy

Python (Original):

```

# Fast tests on commit
- id: pytest-fast
  args: [-m, "not slow", --tb=short, -q]
  stages: [commit]

# All tests on push
- id: pytest-all
  args: [--tb=short]
  stages: [push]

```

TypeScript (Ours):

```

# pre-commit: Fast type check only
tsc --noEmit

# pre-push: Full build + tests
pnpm run build && pnpm run test:run

```

Pattern 2: Auto-fix Philosophy

Python (Original):

```

- id: ruff
  args: [--fix, --exit-non-zero-on-fix]

```

TypeScript (Ours):

```

"lint-staged": {
  "/*.ts": ["eslint --fix", "prettier --write"]
}

```

```
}
```

Both auto-fix issues when possible, block when manual fixes needed.

Pattern 3: Strict Type Checking

Python (Original):

```
- id: mypy
args: [--strict, --ignore-missing-imports]
```

TypeScript (Ours):

```
tsc --noEmit --strict
```

Both enforce strict type safety from the start.

Pattern 4: Security-First

Python (Original):

```
- id: detect-secrets
args: [--baseline, '.secrets.baseline']
```

TypeScript (Ours):

```
# Basic secret detection
git diff --cached | grep -E '\.(env|key|pem)$'

# Dependency audit
pnpm audit --audit-level=high
```

Both prevent secrets from entering version control.

Performance

Stage	Checks	Time	Frequency
pre-commit	Lint + Format + Types	2-10s	Every commit
pre-push	Full Lint + Build + Security	30-60s	Every push
CI/CD	All + Tests + Deploy	2-5min	Every push (remote)

Total time saved: By catching issues locally (10s) vs waiting for CI/CD (5min), you save ~**4min 50s per failed commit**.

Future Enhancements (TODO)

Based on remaining Chris Hayuk patterns:

1. Test Suite Integration

```
# pre-push hook (add this)
pnpm run test:run || {
  echo "    Tests failed"
  exit 1
}
```

2. Coverage Threshold

```
# pre-push hook (add this)
pnpm run test:coverage --coverage.threshold.statements=80 || {
  echo "    Coverage below 80%"
  exit 1
}
```

3. Advanced Secret Detection

```
# Install gitleaks or detect-secrets
pnpm add -D @secretlint/quick-start

# .husky/pre-commit
pnpm exec secretlint "**/*"
```

4. Markdown Linting

```
# Already have prettier for markdown
# Optional: Add markdownlint for stricter rules
pnpm add -D markdownlint-cli
```

5. Commit Message Linting

```
# Install commitlint
pnpm add -D @commitlint/cli @commitlint/config-conventional

# .husky/commit-msg
pnpm exec commitlint --edit $1
```

Comparison: Local Hooks vs CI/CD

Feature	Local Hooks (Husky)	CI/CD (GitLab)
Speed	2-60 seconds	2-5 minutes
Feedback	Immediate	After push
Auto-fix	Yes	No
Cost	Free (local CPU)	Uses CI/CD minutes
Skippable	Yes (–no-verify)	No
Coverage	Basic quality	Full quality + deploy

Best Practice: Use both! Hooks catch 90% of issues locally (fast), CI/CD catches the remaining 10% and handles deployment.

References

- **Chris Hayuk's Templates:** <https://github.com/chrishayuk/vibe-coding-templates>
- **Husky Docs:** <https://typicode.github.io/husky/>
- **lint-staged:** <https://github.com/lint-staged/lint-staged>
- **CI/CD Patterns:** [docs/CI-CD-EXPERT-PATTERNS.md](#)

FAQ

Q: Can I bypass hooks?

A: Yes, use `git commit --no-verify` or `git push --no-verify`, but **only in emergencies**. CI/CD will still catch issues.

Q: Why does pre-commit run on ALL files for type checking?

A: TypeScript requires full project context for accurate type checking. We only lint/format staged files to save time.

Q: What if I disagree with an ESLint rule?

A: Discuss with team, then disable in `.eslintrc` if appropriate. Don't skip hooks.

Q: Do hooks slow down my workflow?

A: No! 2-10 seconds on commit vs 5+ minutes waiting for CI/CD feedback = **massive time savings**.

Q: What's the difference from CI/CD?

A: Local hooks are **fast feedback** (seconds). CI/CD is **comprehensive validation** (minutes) + deployment.